# PhantasiAI

## Software Architecture Document

**Version 0.1**

# Revision History

| Date | Version | Description | Author |
|---|---|---|---|
| 2025 - 07 - 03 | 0.1 | Initial template creation and first content draft<br>what's highlighted are things I'm not sure about | Skander Hellal |
| | | | |
| | | | |
| | | | |
| | | | |

# Table of Content


**1. Introduction** **4**
**2. Objectives and Architectural Constraints** **4**
**3. Use Case View** **5**
1. General App Use Case Diagram 5
2. App Settings Configuration Use Case Diagram 5
3. Graph Interaction Use Case Diagram 6
4. Chat Interaction Use Case Diagram 6
**4. Logic View** **7**
Class Diagram 7
**5. Process View** **8**
**6. Deployment View** **8**
**7. Size and Performance** **8**

# Software Architecture Document

## 1. Introduction

This document provides a high-level software architecture description for the PhantasiAI system. Using multiple architectural diagrams that represent distinct software views, it explains the key design aspects that structure the system. In addition to justifying architectural choices with respect to non-functional requirements, this document also outlines the direct links between the system architecture and its functional requirements.

Many architectural decisions were driven by expected features of the system, such as real-time EMG data acquisition, offline session replay, embedded AI interaction, and a fully touch-based interface. As such, the proposed architecture addresses both general technical constraints and concrete needs required for the software's correct and efficient operation.

## 2. Objectives and Architectural Constraints

This document presents a general description of the software architecture for the PhantasiAI system. It is supported by various architectural views that clarify how the system is structured, how different modules interact, and how responsibilities are distributed across services and threads. Each architectural choice is justified in relation to the requirements of the project.

A significant portion of the design decisions directly responds to functional requirements. For instance, the ability to operate in both live and offline modes, perform real-time EMG visualization, and allow AI-based interpretation via a local model has heavily influenced the modular structure of the system. This led to a separation of concerns across specialized modules such as data acquisition, visualization, AI interaction, and logging that communicate within a lightweight, event-driven architecture. The Raspberry Pi acts as the central processing unit, handling all tasks locally without the need for internet access.

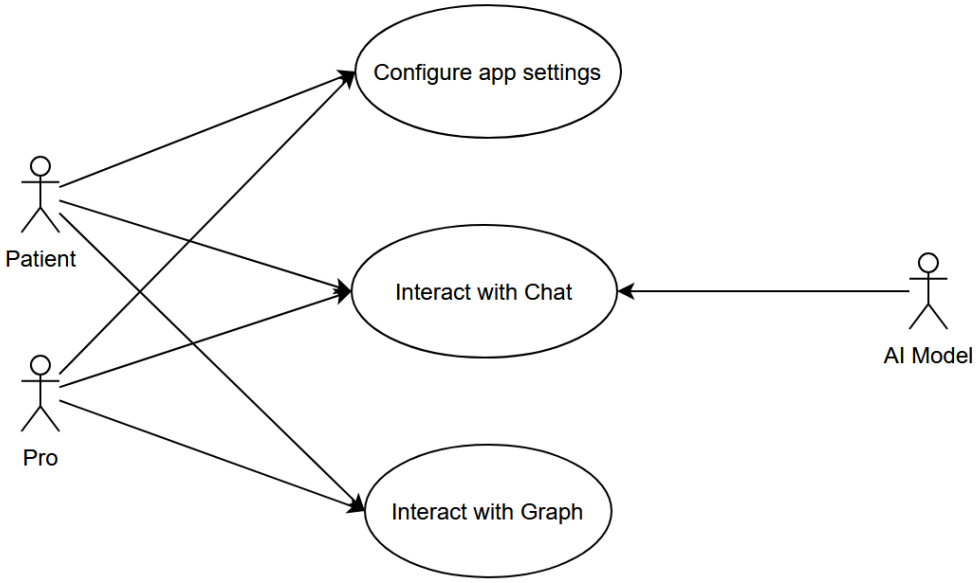
Additionally, the architecture was designed to meet non-functional requirements such as performance, maintainability, and usability under hardware constraints. Given the limited resources of the Raspberry Pi (low RAM, no GPU), the system relies on efficient libraries like PyQt5, PyQtGraph, and NumPy. The application is entirely self-contained and avoids resource-heavy operations, ensuring smooth performance even on minimal hardware.

Touchscreen interaction and accessibility were also key design considerations, enabling the system to be used without a keyboard or mouse. Moreover, by complying with biomedical data standards and ensuring clear data separation, the architecture remains compliant with best practices in clinical and research environments.

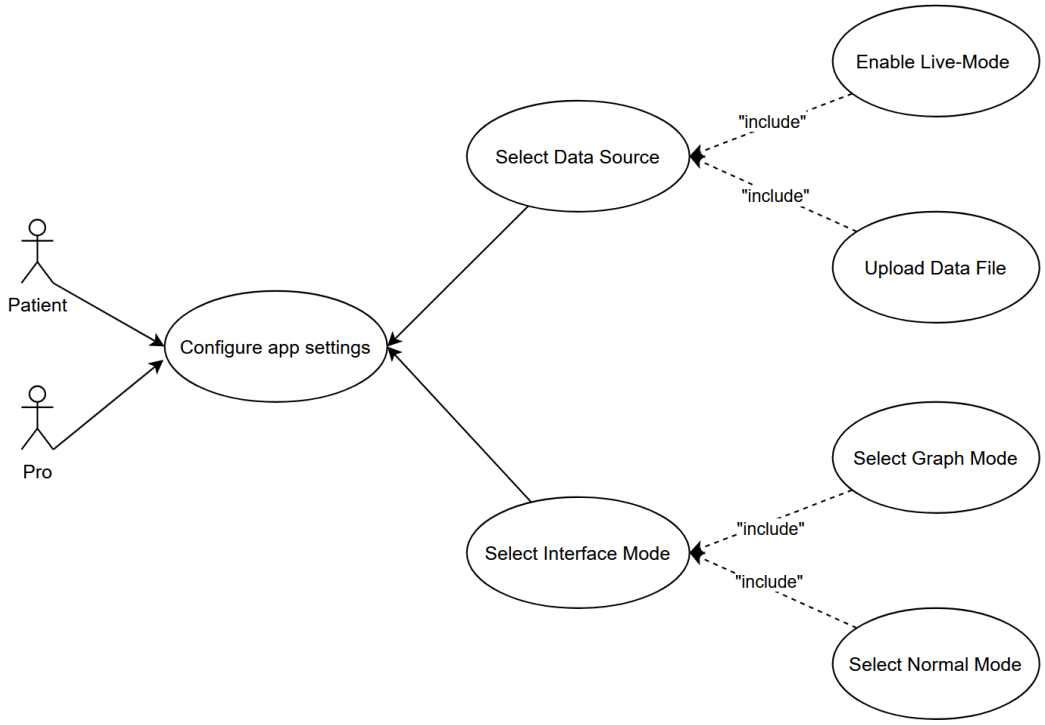
Each architectural view in this document demonstrates how both functional and non-functional requirements shaped our technical decisions leading to a system that is reliable, extensible, and aligned with real-world usage needs.

## 3. Use Case View

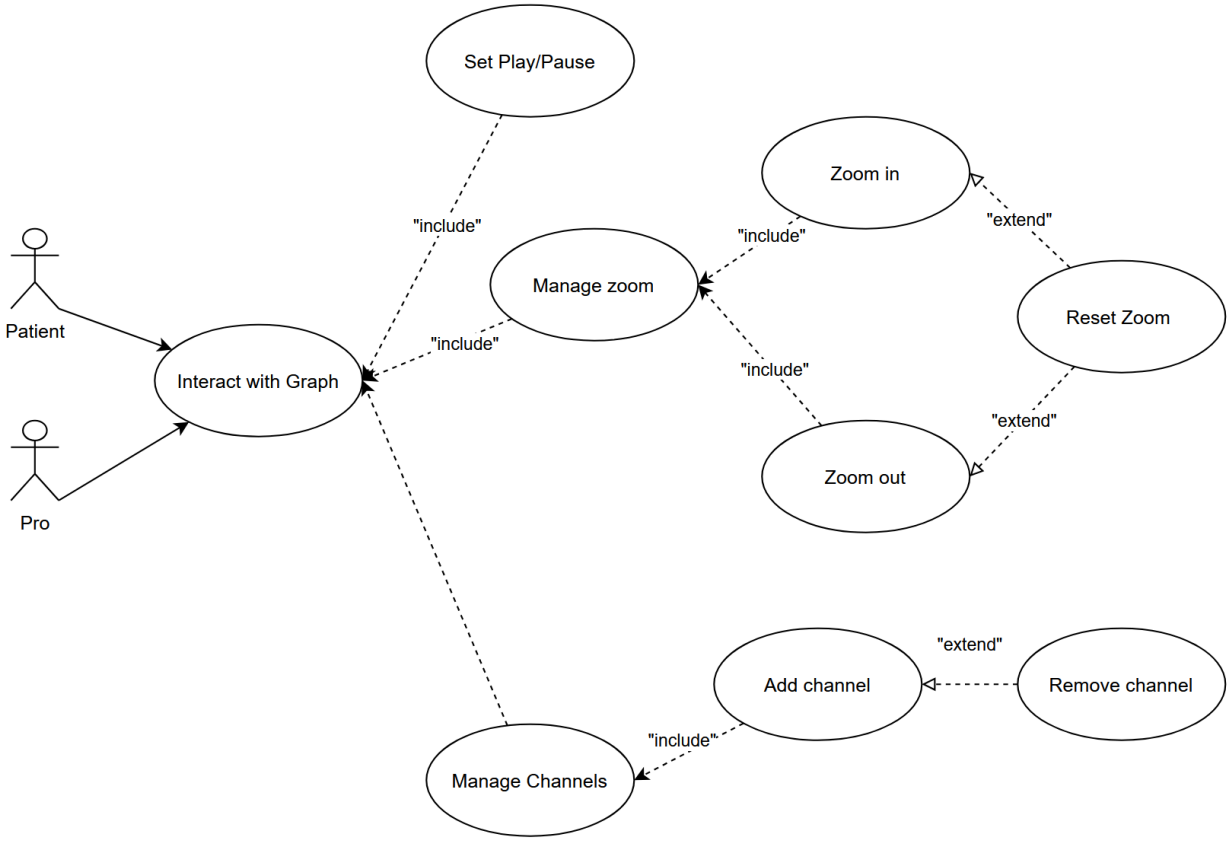
*1. General App Use Case Diagram*



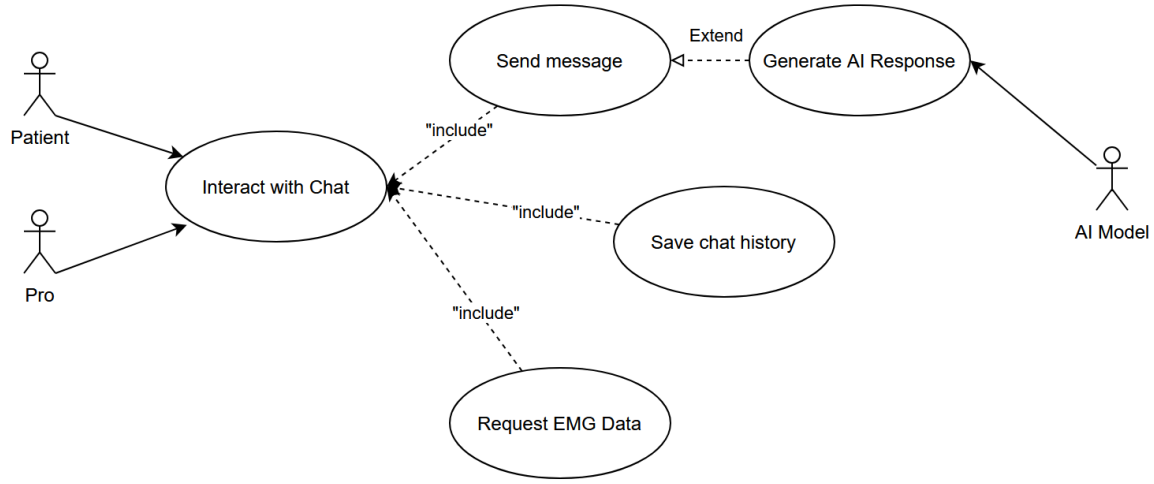

*2. App Settings Configuration Use Case Diagram*



*3. Graph Interaction Use Case Diagram*

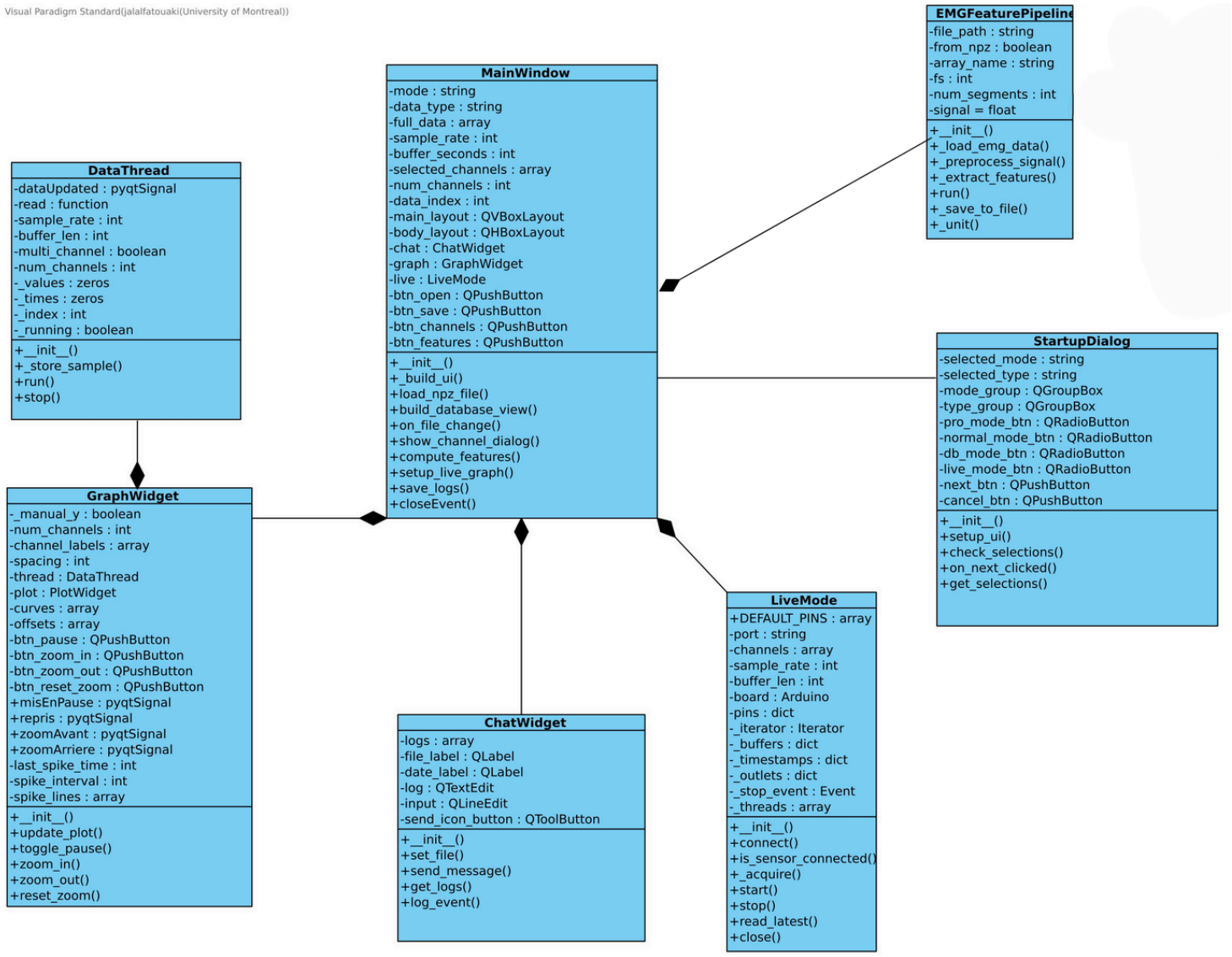



*4. Chat Interaction Use Case Diagram*



## 4. Logic View

*Class Diagram*




| **Main Window** |
|---|
| This class is the central controller of the application. It manages the overall layout and coordination of components such as the graph display, chat system, and live data mode. It handles the loading of files, user interaction with buttons, and the orchestration of feature extraction processes. It serves as the main entry point for the user interface and ensures smooth communication between different modules of the application. |

| **DataThread** |
|---|
| This class is responsible for managing data acquisition in a separate thread. It reads and stores signal data, updates values in real time, |

| |
|---|
| and communicates with the GUI using signals. It supports both single and multi-channel acquisition modes and handles timing and buffer-related operations. It ensures that data is continuously collected and ready for visualization or analysis. |

| **GraphWidget** |
|---|
| This class handles the visualization of EMG signals. It provides interactive tools to pause, zoom, and reset the graphical display. It manages the plotting of multiple channels and updates the graph in response to new data. It plays a critical role in enabling users to monitor and analyze the signal flow in real time. |

| **ChatWidget** |
|---|
| This class manages a simple chat or log interface embedded in the application. It allows for the sending and displaying of messages, including system logs and user input. It helps maintain a record of important events and facilitates communication within the application context. |

| **StartupDialog** |
|---|
| This class is responsible for the initial configuration dialog. It allows users to select the desired mode and data type before launching the main interface. It validates user selections and determines the application's initial state. It ensures a guided startup process adapted to different user scenarios (e.g., live, database, or processing modes). |

| **LiveMode** |
|---|
| This class handles real-time signal acquisition from hardware devices such as Arduino boards. It manages connection states, buffers, timestamps, and data reading. It runs acquisition tasks on multiple threads and provides methods to check sensor status and retrieve the latest samples. It enables live monitoring and processing of biosignals during a session. |

| **EMGFeaturePipeline** |
|---|
| This class manages the overall pipeline for EMG feature extraction. It loads signal data, preprocesses it, segments it, and extracts various features. It also supports saving results to a file. This class provides high-level orchestration of the data processing flow and acts as a bridge between raw EMG signals and analytical outputs. |

## 5. Process View

TODO

## 6. Deployment View

TODO

## 7. Size and Performance

The system is designed to handle real-time EMG signal acquisition and processing, supporting multiple channels (typically 4 to 8). Each channel can record thousands of samples per second depending on the

sampling rate, resulting in moderate memory usage during live sessions. Buffer sizes are optimized to store several seconds of data per channel, leading to an estimated memory footprint between 500 KB and 2 MB (GUESSED NUMBERS). Additionally, logs and signal data are saved in compressed .npz format, with typical session files ranging from RANGE OF FILE SIZE depending on duration.

To ensure a responsive user experience, the system separates UI, acquisition, and processing tasks across multiple threads. Real-time data is streamed with a low-latency design, targeting an update delay of under 100 milliseconds for live graphing. Feature extraction operations, such as computing signal metrics or segment analysis, are highly optimized and complete in less than one second for short signals. Startup time for the application remains minimal, and file loading is designed to be efficient, even with moderately large datasets.

The performance of the system depends on Rapsberry Pi Configuration It should be able to use Python libraries like NumPy and PyQt5 for efficient numerical and GUI operations. While the architecture supports scaling to more channels or advanced processing pipelines, long-duration sessions or very high sampling rates may require additional memory or CPU resources to maintain responsiveness.